

Multi-armed bandits:

Reinforcement Learning - Lecture 5

Felipe Meneguzzi
`felipe.meneguzzi@pucrs.br`

PUCRS

Material adapted from:
Martha and Adam White - University of Alberta/Coursera
Sutton and Barto - Reinforcement Learning: An Introduction (2nd ed.)

Outline

- 1 Recap
- 2 Multi-armed Bandits
- 3 Incremental Updates
- 4 Exploration in Bandits
- 5 Gradient-Bandit Algorithms
- 6 Comparison

- Last lecture:
 - Planning by dynamic programming
 - Solving a **known** MDP
- This lecture (Sutton and Barto 2.1-2.8)
 - **Bandit problems**
 - Exploration and exploitation in single decisions
- Next lecture
 - **Prediction**
 - Monte-Carlo and Temporal Difference Learning

Recap

- We understand how to plan in stochastic environments for both short-term and long-term goals via:
 - MDP formalization
 - Dynamic Programming algorithms
- These formalisms assume we have both $\mathcal{R}$ and $\mathcal{P}$
- But how can we make decisions when we have neither of these?

Outline

- 1 Recap
- 2 Multi-armed Bandits**
- 3 Incremental Updates
- 4 Exploration in Bandits
- 5 Gradient-Bandit Algorithms
- 6 Comparison

k-Armed Bandit Problem

k-armed Bandit Problem

- Repeated choices among k different options or actions
- Each choice A_t has a reward R_t drawn from a stationary probability distribution; reward is **identically, independently distributed** (i.i.d.)
- Goal is to maximize rewards over a period of time

Thus, the value is the expected reward:

$$\begin{aligned} q_*(a) &\doteq \mathbb{E} [R_t \mid A_t = a] \quad \forall a \in \{1, \dots, k\} \\ &= \sum_r \mathbb{P}[r \mid a] r \end{aligned}$$

And to maximize it over time

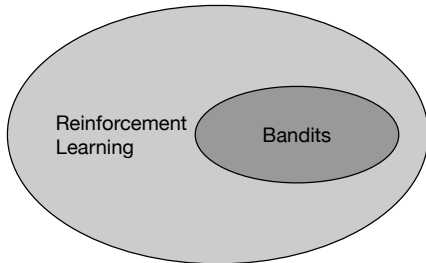
$$\arg \max_a q_*(a)$$



Why bandits?

Why do we have an entire lecture for Bandits problems?

- Bandits problems basically the lotteries we studied in Lecture 2
- Simplest setting to analyze reward maximization and exploration problems



Computing the expected reward

Consider a doctor with 3 possible treatments to offer a patient.

- Each treatment can either cure the patient ($r = 1$), do nothing ($r = 0$), or make the patient sicker ($r = -1$), with probabilities ($\mathbb{P}[r \mid \text{drug}]$) as follows:

d_1  $\mathbb{P}[r = 1 \mid d = d_1] = 0.1$, $\mathbb{P}[r = 0 \mid d = d_1] = 0.8$, and $\mathbb{P}[r = -1 \mid d = d_1] = 0.1$

d_2  $\mathbb{P}[r = 1 \mid d = d_2] = 0.3$, $\mathbb{P}[r = 0 \mid d = d_2] = 0.1$, and $\mathbb{P}[r = -1 \mid d = d_2] = 0.6$

d_3  $\mathbb{P}[r = 1 \mid d = d_3] = 0.3$, $\mathbb{P}[r = 0 \mid d = d_3] = 0.5$, and $\mathbb{P}[r = -1 \mid d = d_3] = 0.2$

Computing the expected reward

Consider a doctor with 3 possible treatments to offer a patient.

- Each treatment can either cure the patient ($r = 1$), do nothing ($r = 0$), or make the patient sicker ($r = -1$), with probabilities ($\mathbb{P}[r \mid \text{drug}]$) as follows:

d_1  $\mathbb{P}[r = 1 \mid d = d_1] = 0.1$, $\mathbb{P}[r = 0 \mid d = d_1] = 0.8$, and $\mathbb{P}[r = -1 \mid d = d_1] = 0.1$

d_2  $\mathbb{P}[r = 1 \mid d = d_2] = 0.3$, $\mathbb{P}[r = 0 \mid d = d_2] = 0.1$, and $\mathbb{P}[r = -1 \mid d = d_2] = 0.6$

d_3  $\mathbb{P}[r = 1 \mid d = d_3] = 0.3$, $\mathbb{P}[r = 0 \mid d = d_3] = 0.5$, and $\mathbb{P}[r = -1 \mid d = d_3] = 0.2$

Compute q_* for all drug options and help the doctor choose:

Computing the expected reward

Consider a doctor with 3 possible treatments to offer a patient.

- Each treatment can either cure the patient ($r = 1$), do nothing ($r = 0$), or make the patient sicker ($r = -1$), with probabilities ($\mathbb{P}[r \mid \text{drug}]$) as follows:

d_1  $\mathbb{P}[r = 1 \mid d = d_1] = 0.1$, $\mathbb{P}[r = 0 \mid d = d_1] = 0.8$, and $\mathbb{P}[r = -1 \mid d = d_1] = 0.1$

d_2  $\mathbb{P}[r = 1 \mid d = d_2] = 0.3$, $\mathbb{P}[r = 0 \mid d = d_2] = 0.1$, and $\mathbb{P}[r = -1 \mid d = d_2] = 0.6$

d_3  $\mathbb{P}[r = 1 \mid d = d_3] = 0.3$, $\mathbb{P}[r = 0 \mid d = d_3] = 0.5$, and $\mathbb{P}[r = -1 \mid d = d_3] = 0.2$

Compute q_* for all drug options and help the doctor choose:

d_1  $q(d = d_1) = (0.1 * 1) + (0.8 * 0) + (0.1 * -1) = 0$

d_2  $q(d = d_2) = (0.3 * 1) + (0.1 * 0) + (0.6 * -1) = -0.3$

d_3  $q(d = d_4) = (0.3 * 1) + (0.5 * 0) + (0.2 * -1) = 0.1$

Learning Action Values

Sample-Average Method

$$\begin{aligned} Q_t(a) &\doteq \frac{\text{sum of rewards for } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} \\ &= \frac{\sum_{i=1}^{t-1} R_i}{t-1} \end{aligned}$$

The sample-average estimates converge to the true values

If the action is taken an infinite number of times

$$\lim_{N_t(a) \rightarrow \infty} Q_t(a) = q_*(a)$$

Example

$$\frac{\sum_{i=1}^{t-1} R_i}{t-1}$$



$$Q_0(\text{pill bottle}) = 0$$



$$Q_0(\text{pill bottle with red}) = 0$$



$$Q_0(\text{pill bottle with green}) = 0$$

Example

$$\frac{\sum_{i=1}^{t-1} R_i}{t-1}$$



1

$$Q_1(\text{pill}) = 1$$



0

$$Q_1(\text{pill}) = 0$$



0

$$Q_1(\text{pill}) = 0$$



Example

$$\frac{\sum_{i=1}^{t-1} R_i}{t-1}$$



1

0

$$Q_2(\text{pill bottle}) = 0.5$$



0

0

$$Q_2(\text{pill bottle}) = 0$$



0

0

$$Q_2(\text{pill bottle}) = 0$$



Example

$$\frac{\sum_{i=1}^{t-1} R_i}{t-1}$$



1

0

0

$$Q_3(\text{pill}) = 0.5$$



0

0

-1

$$Q_3(\text{pill}) = -1$$



0

0

0

$$Q_3(\text{pill}) = 0$$



Example

$$\frac{\sum_{i=1}^{t-1} R_i}{t-1}$$



1

0

0

0

$$Q_4(\text{pill bottle with white label}) = 0.5$$



0

0

-1

0

$$Q_4(\text{pill bottle with red label}) = -1$$



0

0

0

1

$$Q_4(\text{pill bottle with green label}) = 1$$



Example

$$\frac{\sum_{i=1}^{t-1} R_i}{t-1}$$



1
0
0
0
0

$$Q_5(\text{pill}) = 0.5$$



0
0
-1
0
0

$$Q_5(\text{pill}) = -0.5$$



0
0
0
1
0

$$= Q_5(\text{pill}) = 1$$



Outline

- 1 Recap
- 2 Multi-armed Bandits
- 3 Incremental Updates**
- 4 Exploration in Bandits
- 5 Gradient-Bandit Algorithms
- 6 Comparison

From averaging to a learning rule

- To simplify notation, we focus on one action

We consider only its rewards, and its estimate after $n + 1$ rewards:

$$Q_n \doteq \frac{R_1 + R_2 + \cdots + R_n}{n - 1}$$

- How can we do this incrementally (without storing all rewards)?

From averaging to a learning rule

- To simplify notation, we focus on one action

We consider only its rewards, and its estimate after $n + 1$ rewards:

$$Q_n \doteq \frac{R_1 + R_2 + \cdots + R_n}{n}$$

- How can we do this incrementally (without storing all rewards)?
- Could we store a running sum and count (and divide), or equivalently:

$$Q_{n+1} = Q_n + \frac{1}{n+1} [R_{n+1} - Q_n]$$

Incremental Update Rule

$$Q_{n+1} = \frac{1}{n} \sum_{i=1}^n R_i$$

Incremental Update Rule

$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\ &= \frac{1}{n} (R_n + \sum_{i=1}^{n-1} R_i) \end{aligned}$$

Incremental Update Rule

$$\begin{aligned}Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\&= \frac{1}{n} (R_n + \sum_{i=1}^{n-1} R_i) \\&= \frac{1}{n} (R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i)\end{aligned}$$

Incremental Update Rule

$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\ &= \frac{1}{n} (R_n + \sum_{i=1}^{n-1} R_i) \\ &= \frac{1}{n} (R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i) \end{aligned} \qquad = \frac{1}{n} (R_n + (n-1) Q_n)$$

Recall: $Q_n = \frac{1}{n-1} \sum_{i=1}^{n-1} R_i$

Incremental Update Rule

$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\ &= \frac{1}{n} (R_n + \sum_{i=1}^{n-1} R_i) \\ &= \frac{1}{n} (R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i) \end{aligned}$$
$$\begin{aligned} &= \frac{1}{n} (R_n + (n-1) Q_n) \\ &= \frac{1}{n} (R_n + n Q_n - Q_n) \end{aligned}$$

Recall: $Q_n = \frac{1}{n-1} \sum_{i=1}^{n-1} R_i$

Incremental Update Rule

$$\begin{aligned}Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\&= \frac{1}{n} (R_n + \sum_{i=1}^{n-1} R_i) \\&= \frac{1}{n} (R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i)\end{aligned}$$

$$\begin{aligned}&= \frac{1}{n} (R_n + (n-1) Q_n) \\&= \frac{1}{n} (R_n + n Q_n - Q_n) \\&= Q_n + \frac{1}{n} (R_n - Q_n)\end{aligned}$$

Recall: $Q_n = \frac{1}{n-1} \sum_{i=1}^{n-1} R_i$

Incremental Update Rule

We will see this form of update throughout this course:

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} [\text{Target} - \text{OldEstimate}]$$

$$Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n)$$

$$Q_{n+1} = Q_n + \alpha_n(R_n - Q_n)$$

Incremental Update Rule

We will see this form of update throughout this course:

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} [\text{Target} - \text{OldEstimate}]$$

$$Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n)$$

$$Q_{n+1} = Q_n + \alpha_n(R_n - Q_n)$$

We will see what α means in a moment

Outline

- 1 Recap
- 2 Multi-armed Bandits
- 3 Incremental Updates
- 4 Exploration in Bandits**
- 5 Gradient-Bandit Algorithms
- 6 Comparison

Exploration on k -armed bandits

- Remember that the reward depends only on the action taken

$$q_*(a) \doteq \mathbb{E} [R_t \mid A_t = a] \quad \forall a \in \{1, \dots, k\}$$

- In reality: these true values are **unknown**. The distribution is **unknown**
- Nevertheless, must maximize total reward, doing both
 - try actions to learn their values (explore)
 - prefer those actions that appear best (exploit)

Greedy Exploitation Example

$$\frac{\sum_{i=1}^{t-1} R_i}{t-1}$$



$$Q_0(\text{slot machine}) = 0$$



$$Q_0(\text{slot machine}) = 0$$



$$Q_0(\text{slot machine}) = 0$$

Greedy Exploitation Example

$$\frac{\sum_{i=1}^{t-1} R_i}{t-1}$$



$$1$$
$$Q_1(\text{slot}) = 1$$



$$0$$
$$Q_1(\text{slot}) = 0$$



$$0$$
$$Q_1(\text{slot}) = 0$$



The Exploration/Exploitation Dilemma

- Suppose you have estimates:

$$Q_t(a) \approx q_*(a), \forall a$$

- Define the **greedy action** at time t as

$$A_t^* \doteq \arg \max_a Q_t(a)$$

- If $A_t = A_t^*$ then you are **exploiting**
If $A_t \neq A_t^*$ then you are **exploring**
- You cannot do both, but you need to do both
- You can never stop exploring, but maybe you should explore less with time. Or maybe not.

ϵ -Greedy Action Selection

- In greedy action selection, you always exploit
- In ϵ -greedy, you are usually greedy, but with some probability ϵ you instead pick an action at random (possibly the greedy action again)
- Possibly the simplest way to balance exploration and exploitation

A simple bandit algorithm

ϵ -greedy Bandit Algorithm

for $a = 1$ to $a = k$ **do**

▷ Initialize tables

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

end for

while True **do**

$A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{with probability } 1 - \epsilon \\ \text{random action} & \epsilon \end{cases}$

$R \leftarrow \text{bandit}(A)$

$N(a) \leftarrow N(a) + 1$

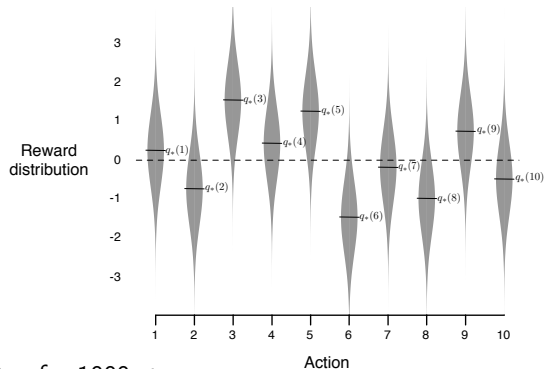
$Q(a) \leftarrow Q(a) + \frac{1}{N(a)} [R - Q(a)]$

end while

The 10-armed testbed

- Testbed for exploration functions
- Each task draws different values for q_*
- Tasks run for 100 steps/epochs

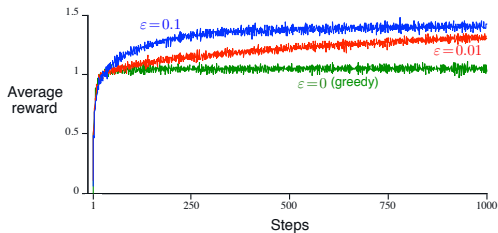
$$q_*(a) \sim \mathcal{N}(0, 1)$$
$$R_t \sim \mathcal{N}(q_*(a), 1)$$



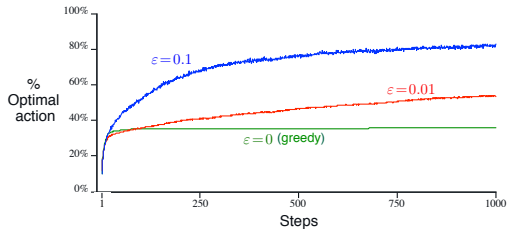
Run for 1000 steps

Repeat 2000 times with different bandit tasks

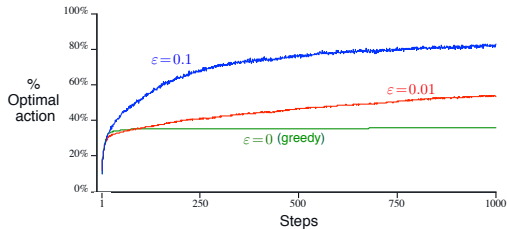
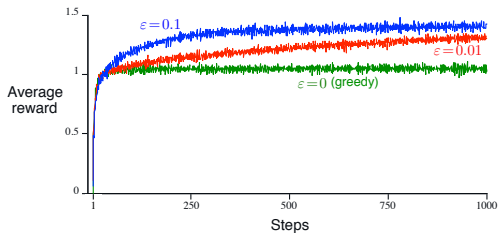
ϵ -greedy Methods on the 10-armed testbed



- Greedy selection improves rewards slightly faster (in the beginning)

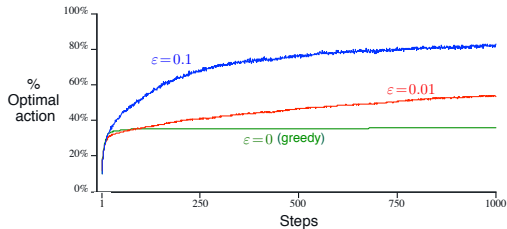
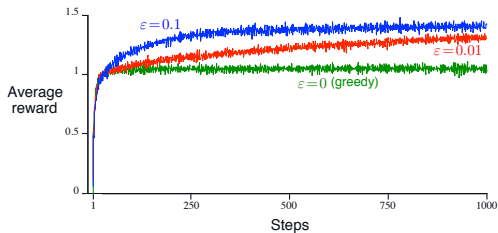


ϵ -greedy Methods on the 10-armed testbed



- Greedy selection improves rewards slightly faster (in the beginning)
- Plateaus at reward ≈ 1 , but $q_* \approx 1.55$
- Greedy reward stuck on suboptimal actions

ϵ -greedy Methods on the 10-armed testbed



- Greedy selection improves rewards slightly faster (in the beginning)
- Plateaus at reward ≈ 1 , but $q_* \approx 1.55$
- Greedy reward stuck on suboptimal actions
- Smaller ϵ ensure higher rewards, larger values ensure faster convergence

Tracking a Non-stationary Problem

- What if the true action values change (slowly) over time?

Tracking a Non-stationary Problem

- What if the true action values change (slowly) over time?
 - problem becomes **nonstationary**
- In this case, why are sample averages are not a good idea?

Tracking a Non-stationary Problem

- What if the true action values change (slowly) over time?
 - problem becomes **nonstationary**
- In this case, why are sample averages are not a good idea?
- Better to use an “exponential, recency-weighted average”:

$$\begin{aligned}Q_{n+1} &= Q_n + \alpha [R_n - Q_n] \\&= (1 - \alpha)^n Q_1 + \sum_{i=1}^n \alpha (1 - \alpha)^{n-i} R_i\end{aligned}$$

where α is a constant, **step-size parameter**, $0 < \alpha \leq 1$

- There is bias due to Q_1 that becomes smaller over time

Tracking a Non-stationary Problem (2)

Convergence conditions for stochastic approximation

- To assure convergence with probability 1:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

Tracking a Non-stationary Problem (2)

Convergence conditions for stochastic approximation

- To assure convergence with probability 1:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

- e.g. $\alpha_n = \frac{1}{n}$

Tracking a Non-stationary Problem (2)

Convergence conditions for stochastic approximation

- To assure convergence with probability 1:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

- e.g. $\alpha_n = \frac{1}{n}$
- not $\alpha_n = \alpha$, or $\alpha_n = \frac{1}{n^2}$ (why?)

Tracking a Non-stationary Problem (2)

Convergence conditions for stochastic approximation

- To assure convergence with probability 1:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \quad \text{and} \quad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

- e.g. $\alpha_n = \frac{1}{n}$
- not $\alpha_n = \alpha$, or $\alpha_n = \frac{1}{n^2}$ (why?)
- If $\alpha_n = n^{-p}$, $p \in (0, 1)$, then convergence is at optimal rate $O(1/\sqrt{n})$

Optimistic Initial Values

- All methods so far are biased on initial value $Q_1(a)$
So far we initialize $Q_1(a) = 0$
- What happens if we initialize values **optimistically**,
.e.g $Q_1(a) \leftarrow 5$

Optimistic Initial Values

- All methods so far are biased on initial value $Q_1(a)$
So far we initialize $Q_1(a) = 0$
- What happens if we initialize values **optimistically**,
.e.g $Q_1(a) \leftarrow 5$
 - Creates an **incentive** to explore

Optimistic Initial Values

- All methods so far are biased on initial value $Q_1(a)$
So far we initialize $Q_1(a) = 0$
- What happens if we initialize values **optimistically**,
.e.g $Q_1(a) \leftarrow 5$
 - Creates an **incentive** to explore
- More parameters to set the algorithm

Optimistic Initial Values (Example)



$$Q_0(\text{pill}) = 5$$



$$Q_0(\text{red pill}) = 5$$



$$Q_0(\text{green pill}) = 5$$

For $\alpha = 0.5$

Optimistic Initial Values (Example)



$$^1 Q_1(\text{slot machine}) = 3$$



$$^0 Q_1(\text{slot machine with red}) = 5$$



$$^0 Q_1(\text{slot machine with green}) = 5$$

For $\alpha = 0.5$



$$R_1 = 1$$

Optimistic Initial Values (Example)



1

0

$$Q_2(\text{pill}) = 3$$



0

-1

$$Q_2(\text{pill}) = 2$$



0

0

$$Q_2(\text{pill}) = 5$$

For $\alpha = 0.5$



$$R_2 = -1$$

Optimistic Initial Values (Example)



1

0

0

$$Q_3(\text{pill}) = 3$$



0

-1

0

$$Q_3(\text{pill}) = 2$$



0

0

1

$$Q_3(\text{pill}) = 3$$

For $\alpha = 0.5$



$$R_3 = 1$$

Optimistic Initial Values (Example)



1

0

0

0

$$Q_4(\text{slot}) = 3$$



0

-1

0

0

$$Q_4(\text{slot}) = 2$$



0

0

1

1

$$Q_4(\text{slot}) = 2$$

For $\alpha = 0.5$



$$R_4 = 1$$

Optimistic Initial Values (Example)



1

0

0

0

0

$$Q_5(\text{pill}) = 1.5$$



0

-1

0

0

0

$$Q_5(\text{pill}) = 2$$



0

0

1

1

0

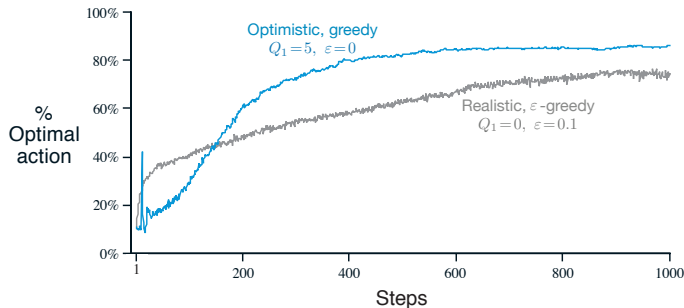
$$Q_5(\text{pill}) = 2$$

For $\alpha = 0.5$



$R_5 = 0$

Optimistic Initial Values



- Quickly explores all actions biased towards unexplored ones
- No residual randomness like ϵ -greedy
- Effective trick in **stationary** problems
- Limitations:
 - Not suitable for **non-stationary** problems
 - **How do we set** the optimistic initial value?

Upper-Confidence-Bound Action Selection

- Exploration required because of uncertainty about estimated Q
 - ϵ -greedy explores indiscriminately
 - Optimistic initialization limited to initial steps
- Upper confidence bound (UCB) selects actions based on the **maximal expected value** of actions

$$A_t \doteq \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

c controls the degree of exploration

- As t increases (without a being selected) incentive to select a
- As $N_t(a)$ (with a selected) uncertainty decreases

Example of UCB



$$Q_0(\text{white pill}) = 0$$



$$Q_0(\text{red pill}) = 0$$



$$Q_0(\text{green pill}) = 0$$

For $c = 2, \alpha = 1$

Example of UCB



$$Q_1(\text{state 1}) = 1$$

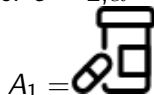


$$Q_1(\text{state 2}) = 0$$



$$Q_1(\text{state 3}) = 0$$

For $c = 2, \alpha = 1$



$$A_1 = \text{state 1}$$
$$R_1 = 1$$

Example of UCB



1

0

$$Q_2(\text{slot}) = 1$$



0

-1

$$Q_2(\text{slot}) = -1$$



0

0

$$Q_2(\text{slot}) = 0$$

For $c = 2, \alpha = 1$
 $0 + 2\sqrt{\frac{\ln 2}{1}} = 1.7$



$$A_2 =$$

$$R_2 = -1$$

Example of UCB



1

0

0

$$Q_3(\text{slot machine}) = 1$$



0

-1

0

$$Q_3(\text{slot machine}) = -1$$



0

0

0

$$Q_3(\text{slot machine}) = 0$$

For $c = 2, \alpha = 1$
 $0 + 2\sqrt{\frac{\ln 3}{1}} = 2.1$



$A_3 =$

$R_3 = 0$

Example of UCB



1
0
0
0

$$Q_4(\text{white pill}) = 0$$



0
-1
0
0

$$Q_4(\text{red pill}) = -1$$



0
0
0
0

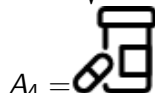
$$Q_4(\text{green pill}) = 0$$

For $c = 2, \alpha = 1$

$$\text{white pill} \rightarrow 1 + 2\sqrt{\frac{\ln 4}{2}} = 2.7$$

$$\text{red pill} \rightarrow -1 + 2\sqrt{\frac{\ln 4}{2}} = 0.7$$

$$\text{green pill} \rightarrow 0 + 2\sqrt{\frac{\ln 4}{2}} = 1.7$$



$$A_4 = \text{white pill}$$

$$R_4 = 0$$

Example of UCB



1
0
0
0
0

$$Q_5(\text{white pill}) = 0$$



0
-1
0
0
0

$$Q_5(\text{red pill}) = -1$$



0
0
0
0
0

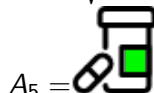
$$Q_5(\text{green pill}) = 0$$

For $c = 2, \alpha = 1$

$$\text{white pill} \rightarrow 0 + 2\sqrt{\frac{\ln 5}{3}} = 1.5$$

$$\text{red pill} \rightarrow -1 + 2\sqrt{\frac{\ln 5}{2}} = 0.8$$

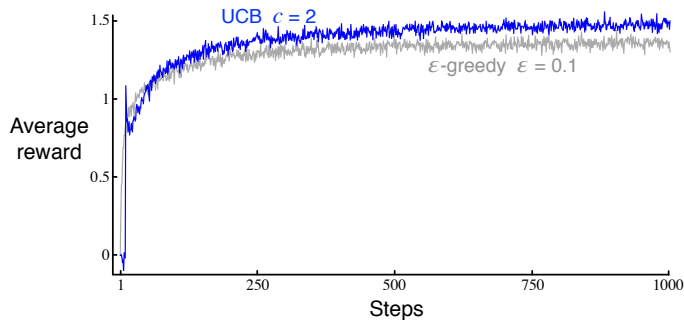
$$\text{green pill} \rightarrow 0 + 2\sqrt{\frac{\ln 5}{2}} = 1.8$$



$$A_5 = \text{green pill}$$

$$R_5 = 0$$

Upper-Confidence-Bound Action Selection



- UCB works well in bandits and tabular RL
- Limitations when dealing with
 - Nonstationary problems
 - Function approximation in RL (non-tabular methods)

Outline

- 1 Recap
- 2 Multi-armed Bandits
- 3 Incremental Updates
- 4 Exploration in Bandits
- 5 Gradient-Bandit Algorithms**
- 6 Comparison

Gradient-Bandit Algorithms

- Methods developed so far try to estimate q_* using Q
- Instead of computing these values, try to learn a preference H_t over actions Akin to policy iteration
- In effect compute a probability to choose an action:

$$\mathbb{P}[A_t = a] \doteq \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}} \doteq \pi_t(a)$$

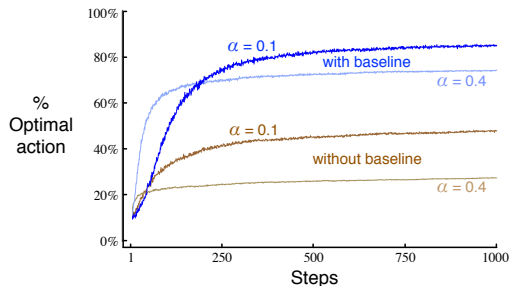
- Initially, all preferences set to zero $H_1(a) = 0$
- Update all H at every interaction

$$\begin{aligned} H_{t+1}(A_t) &\doteq H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)), && \text{and} \\ H_{t+1}(a) &\doteq H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a), && \text{for all } a \neq A_t \end{aligned}$$

Derivation of gradient

See Sutton and Barto 2.8

Gradient-Bandit Algorithms



- $\bar{R}_t = \frac{\sum_{i=1}^t R_i}{t}$ serves as baseline for the rewards
 - If a reward is higher than baseline, increase probability of taking it
 - Decrease probability otherwise
 - All other actions move in the opposite direction
- Baseline impacts performance (no baseline means $\bar{R}_t = 0$)

Outline

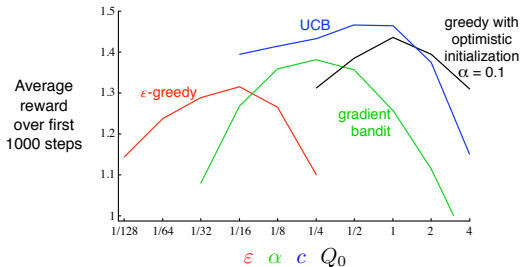
- 1 Recap
- 2 Multi-armed Bandits
- 3 Incremental Updates
- 4 Exploration in Bandits
- 5 Gradient-Bandit Algorithms
- 6 Comparison**

Conclusions

- Today we explored simple methods for almost-RL
 - Methods complicated enough
 - We should understand them
- We saw algorithms that learn from evaluative feedback which balance exploration and exploitation
- These algorithms maximize reward by trial and error

Comparison of methods

- We saw several methods to compute policies for bandit problems
- In a parameter study, look for:
 - Performance at best parameter
 - Performance at worst parameter
 - Degradation over time
- These ideas form the basis for the rest of our study of RL



Lecture Summary

- Multi-armed Bandit problems
- Incremental Updates
 - Basic learning rule
- Exploration and Exploitation in Bandits
- Gradient Bandits